

ARM VS RISC-V

MODERN RISC-V ARCHITECTURES

COURSE: COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE

PRESENTED BY:

[Saad mughal - F2023-009]

[Sheikh muhammad ibrahim - F2023-630]

[Ramsha Naveed - F2023-026]

[Muhammad Ismail Rana - F2023-551]



INTRODUCTION TO THE ARCHITECTURES

ARM ARCHITECTURE

Founded: 1983 by Acorn Computers

Current Owner: ARM Holdings (now owned by NVIDIA)

Key Versions:

- ARMv7 (32-bit) - Cortex-A/M/R series
- ARMv8 (64-bit) - A64, AArch32/64
- ARMv9 (Latest) - Enhanced security & ML

RISC-V ARCHITECTURE

Founded: 2010 at UC Berkeley

Current Owner: Open standard by RISC-V International

Key Versions:

- RV32I (32-bit base integer)
- RV64I (64-bit base integer)
- Extensions: M(mul), A(atomic), F(float), D(double)

TARGET DEVICES & MARKETS

ARM Dominance: Smartphones (95% share) • Laptops • Servers (Graviton) • IoT

RISC-V Growth: Academia • Custom SoCs • Edge • Space & Safety-critical apps

INSTRUCTION SET ARCHITECTURE (ISA)

Both are Pure RISC Architectures - Fixed-length instructions • Load-store architecture • Large register files

Feature	ARM	RISC-V
Instruction Formats	Multiple: ARM (32-bit), Thumb (16-bit), Thumb-2 (16/32-bit)	Modular: Base + Extensions (RV32I + M/A/F/D/C)
Base Instructions	~200+ instructions in ARMv8	47 base instructions in RV32I
Addressing Modes	13 addressing modes (complex)	3 addressing modes (simple)
Conditional Execution	Most instructions can be conditional	Branch instructions only

REGISTER FILE DESIGN

ARM REGISTERS

- **ARMv7:** 16 general-purpose registers (R0-R15)
- **ARMv8:** 31 general-purpose registers (X0-X30) + SP
- **Special:** PC, SP, LR (Link Register)
- **NEON:** 32×128-bit vector registers

RISC-V REGISTERS

- **Integer:** 32 general-purpose registers (x0-x31)
- **x0:** Always zero (hardwired)
- **Floating Point:** 32×64-bit FP registers (f0-f31)
- **Vector:** Configurable vector registers (RVV)

MICROARCHITECTURAL DESIGN

PIPELINE ARCHITECTURE

ARM PIPELINE

Complexity: 8-15 stages (implementation dependent)

Fetch

Decode

Rename

Dispatch

Execute

Writeback

Examples: Cortex-A78 (11-stage), Cortex-M4 (3-stage)

RISC-V PIPELINE

Flexibility: 3-7 stages (typically 5-stage classic RISC)

IF

ID

EX

MEM

WB

Examples: SiFive U74 (8-stage), Rocket (5-stage)

EXECUTION CHARACTERISTICS

Feature	ARM	RISC-V
Superscalar	Yes (2-6 way in Cortex-A series)	Implementation dependent (usually scalar)
Out-of-Order	Yes in high-performance cores	Custom implementations only
Branch Prediction	Advanced (hybrid, neural)	Simple to advanced (implementation dependent)
Cache Coherence	AMBA CHI, ACE protocols	Implementation specific

CACHE PHILOSOPHY

ARM

Standardized with coherence

RISC-V

Flexible, tailored designs

PERFORMANCE FACTORS

CLOCK SPEED VS IPC

Feature	ARM	RISC-V
Clock Speed	1.8–3.5 GHz	100 MHz–2.5 GHz
IPC (Instructions)	4–6 (Cortex-X1, A78)	0.5–3+ (custom designs)
Compiler Support	Highly mature (GCC, LLVM)	Rapidly improving (GCC, LLVM)

POWER EFFICIENCY (IOT → HPC)

Domain	ARM Power	RISC-V Power	Winner
IoT Devices	10-100 μW	5-50 μW	RISC-V
Mobile	1-5 W	Limited Support	ARM
HPC	120-280 W	Emerging	ARM

THERMAL MANAGEMENT

ARM

- Dynamic Voltage and Frequency Scaling (DVFS)
- big.LITTLE heterogeneous computing
- Advanced thermal throttling
- Optimized for fanless mobile devices

RISC-V

- Custom designs for specific thermal envelopes
- Minimal die area for simple implementations
- Configurable power management features
- Excellent for space-constrained applications

STRENGTHS AND USE CASES

ARM STRENGTHS

- 30+ years of evolution
- Robust ecosystem & SoC integration
- High performance + power efficiency

RISC-V STRENGTHS

- Open-source & vendor-neutral
- Easy customization and rapid innovation
- Academic-friendly

DOMAIN EXPERTISE

ARM DOMINANCE AREAS

- **Mobile Computing:** 95% smartphone market share, optimized for user experience and battery life
- **Personal Computing:** Apple Silicon revolution, Windows on ARM growth
- **Cloud Computing:** AWS Graviton, Google Axion, Microsoft Cobalt processors
- **Industrial IoT:** Cortex-M series for real-time embedded applications

RISC-V GROWTH AREAS

- **Education:** Preferred architecture for computer architecture courses
- **Research:** Academic projects and novel architecture exploration
- **Security:** Government and military applications requiring open-source verification
- **AI/ML Acceleration:** Custom neural network processors and accelerators
- **Space Applications:** Radiation-hardened custom processors

WEAKNESSES AND LIMITATIONS

ARM LIMITATIONS

- Expensive licensing
- Vendor lock-in
- Complex evolving ISA
- Geopolitical export risks

RISC-V LIMITATIONS

- Immature ecosystem
- Performance lag (vs ARM)
- Extension fragmentation risk
- Limited SoC toolchains
- Debugging/profiling tools are young

MARKET FAILURE ANALYSIS

ARM CHALLENGES

- x86 emulation issues (Windows)
- Late entry in HPC & Gaming

RISC-V BARRIERS

- Enterprise hesitation
- Weak OS/app support
- No standard reference SoCs

REAL-WORLD EXAMPLES

ARM

1

CONSUMER:

- Apple A17 Pro, M3 • Snapdragon 8 Gen 3 • Exynos 2400
- Dimensity 9300 (MediaTek)

2

SERVERS:

- AWS Graviton3/4 • Google Axion • Azure Cobalt • Ampere Altra

Gaming: Switch | **Automotive:** Tesla FSD | **Supercomputing:** Fugaku

RISC-V

1

COMMERCIAL CHIPS:

- SiFive FU740 • Espressif ESP32-C3/C6 • Bouffalo BL602 • Cannaan K210

2

EMERGING USES:

- Alibaba Xuantie • NVIDIA GPU control • WD storage • NASA HPSC

Gaming: Retro emulators | **Automotive:** NXP ECU | **Supercomputing:** EPI Exascale

COMPARATIVE SUMMARY

TECHNICAL SPECIFICATIONS COMPARISON

Feature	ARM	RISC-V	Advantage
Instruction Size	16/32-bit (Thumb/ARM)	16/32-bit (Compressed/Base)	Tie
Code Density	Excellent (Thumb-2)	Good (RVC extension)	ARM
Power Efficiency	Industry-leading	Highly customizable	Context-dependent
Performance/Watt	Optimized for mobile	Optimized for specific use	ARM (mobile)
Licensing	Proprietary (\$\$)	Open source (Free)	RISC-V
Customization	Limited	Unlimited	RISC-V
Ecosystem Maturity	Very mature	Rapidly developing	ARM
Software Support	Comprehensive	Growing	ARM

VECTOR EXTENSIONS (RVV)

Why RVV is Innovative?

Unlike ARM's fixed NEON (128-bit), RVV supports vector-length agnostic programming—write once, run anywhere!

NEON VS RVV

Feature	ARM NEON	RISC-V RVV
Width	Fixed 128-bit	Variable (128-2048+ bits)
Remainders	Manual scaler fallback	Automatically handled
Scalability	Limited	Infinitely scalable

CODE COMPARISON

- **NEON:** Manual 4-lane loop + scalar tail
- **RVV:** Auto-sized loop with vsetvl → zero rewrite for wider vector

RVV IMPACT AREAS

- **HPC:** Simulations, fluid dynamics
- **AI/ML:** Tensor operations
- **DSP:** Audio/video codecs
- **Crypto:** Parallel encryption

CUSTOM INSTRUCTION EXTENSIONS

Why Custom Instructions Matter?

RISC-V allows ISA extension for specific domains. ARM does not.

EXAMPLE: AES ENCRYPTION

- **Standard:** 20+ instructions
- **Custom:** Single aes_round instruction
- **Result:** 20× faster, 95% energy saved

AI/ML MATRIX MULTIPLICATION

- **Standard:** Nested loops
- **Custom:** matrix_multiply_accelerated()
- **Result:** 100× faster inference

COMPARISON

Feature	ARM	RISC-V
Custom Instructions	Not allowed	Supported
Innovation Speed	Slower	Rapid prototyping
Trust	Closed-source	Community-verified

CONCLUSION

ARCHITECTURE VIABILITY

ARM

- Mature, efficient, dominant
- Expected to lead in mainstream for 5–10 years

RISC-V

- Open, flexible, innovative
- Will lead in cost-sensitive, custom, and niche markets

FUTURE TRENDS

2025–2030

- ARM: Expands to laptops/servers
- RISC-V: Breakthrough in IoT/Edge, early server success

2030–2035

- ARM: Slower growth
- RISC-V: High-performance & global adoption

FINAL TAKEAWAYS

In conclusion, both ARM and RISC-V are poised to thrive in distinct domains. While ARM will continue to dominate mainstream markets, RISC-V is accelerating innovation and driving the industry toward a new era of customizable, domain-specific silicon. Rather than replacing one another, these architectures are likely to coexist, with RISC-V's openness pushing ARM to evolve faster. Ultimately, the success of either architecture will depend heavily on the maturity and adaptability of their ecosystems.

THANK YOU

QUESTIONS & DISCUSSION

Key Resources:

- *The RISC-V Reader* – Patterson & Waterman
- *Computer Architecture: A Quantitative Approach* – Hennessy & Patterson
- RISC-V Specifications: riscv.org/specifications
- ARM Documentation: developer.arm.com/documentation

References:

- Dakić et al., *Evaluating ARM and RISC-V for HPC*, Electronics, 2024
doi.org/10.3390/electronics13173494
- LLVM Docs: [RISC-V Vector Extension](#)
- GitHub Docs: [RVV Intrinsics Spec](#)
- Design Reuse: [Extending RISC-V ISA](#)
- arXiv:
 - [Custom Instructions in LLVM](#)
 - [RVV Permutation Optimization](#)
- ACM DL: [RISC-V vs AArch64 Analysis](#)

PRESENTED BY:

[Saad mughal - F2023-009]
[Sheikh Muhammad Ibrahim - F2023-630]
[Ramsha Naveed - F2023-026]
[Muhammad Ismail Rana - F2023-551]